

在 `React.js` 中，組件可以使用函數和類別來定義。

以下是函數和類別組件之間的一些主要差異：

語法：函數組件是一個函數，類別組件是一個類別，它們有不同的語法結構。

差異說明：

1. 語法：`Function Component` 使用箭頭函式定義，而 `Class Component` 則是使用類別的語法。
2. 繼承：`Class Component` 繼承自 `React` 的 `Component` 類別，並使用 `render` 方法來定義元件的內容。而 `Function Component` 則是直接返回 `JSX` 元素。
3. 狀態管理：`Class Component` 具有內部狀態（`state`）的管理能力，可以使用 `this.state` 和 `this.setState` 來處理狀態的更新。`Function Component` 則沒有內建的狀態管理功能，但可以使用 `React` 的 `Hooks`（如 `useState`）來處理狀態。
4. 生命週期方法：`Class Component` 提供了多個生命週期方法（如 `componentDidMount`、`componentDidUpdate` 等），可以在不同階段進行操作。而 `Function Component` 可以使用 `React` 的 `Hooks`（如 `useEffect`）來模擬生命週期方法的行為。
5. 程式碼結構：`Class Component` 需要定義類別和方法，相對而言比較冗長。`Function Component` 無需額外定義類別，只需要一個函式即可，更加簡潔和清晰。

需要注意的是，從 `React 16.8` 版本開始，`Function Component` 引入了 `Hooks`，使得函式組件可以擁有狀態管理和生命週期方法等類似 `Class Component` 的功能。因此，使用 `Hooks` 的 `Function Component` 已經成為 `React` 中最常見的元件定義方式。

以上是 `Function Component` 和 `Class Component` 的差異舉例，您可以根據項目需求和個人偏好選擇使用哪種元件定義方式。

```
import React from 'react';
```

```
const FunctionalComponent = (props) => {  
  return (  
    <div>  
      <h1>Hello, {props.name}!</h1>  
      <p>This is a functional component.</p>  
    </div>  
  );  
};
```

```
export default FunctionalComponent;
```

```
=====
```

```
import React, { Component } from 'react';
```

```
class ClassComponent extends Component {  
  render() {  
    return (  
      <div>  
        <h1>Hello, {this.props.name}!</h1>  
        <p>This is a class component.</p>  
      </div>  
    );  
  }  
}
```

```
export default ClassComponent;
```

在 Class Component 中，我們可以使用 **this.state** 和 **this.setState** 來處理內部的狀態管理。在 **constructor** 中，我們可以初始化狀態（**state**），並在其他方法中使用 **this.setState** 來更新狀態。在 **render** 方法中，我們可以使用 **this.state** 來讀取當前的狀態。

```
import React, { Component } from 'react';
```

```
class ClassComponent extends Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      count: 0,  
    };  
  }  
  
  incrementCount() {
```

```

        this.setState({ count: this.state.count + 1 });
    }

    render() {
        return (
            <div>
                <h1>Count: {this.state.count}</h1>
                <button onClick={() => this.incrementCount()}>Increment</button>
            </div>
        );
    }
}

export default ClassComponent;

```

在 **Function Component** 中，我們可以使用 **React** 的 **Hooks**（如 **useState**）來處理狀態。使用 **useState Hook** 可以定義狀態變數（如 **count**）和更新狀態的函式（如 **setCount**）。在函式組件中，我們可以直接使用狀態變數（如 **count**）和更新狀態的函式（如 **setCount**）來處理狀態的讀取和更新。

總結來說，**Class Component** 具有內部狀態的管理能力，可以使用 **this.state** 和 **this.setState** 來處理狀態的更新。**Function Component** 則可以使用 **React** 的 **Hooks**（如 **useState**）來處理狀態，並且不需要使用 **this** 關鍵字來訪問狀態。這些 **Hooks** 提供了一個簡潔而強大的方式來處理函式組件的狀態管理。

```

import React, { useState } from 'react';

const FunctionalComponent = () => {
    const [count, setCount] = useState(0);

    const incrementCount = () => {
        setCount(count + 1);
    };

    return (
        <div>
            <h1>Count: {count}</h1>

```

```
        <button onClick={incrementCount}>Increment</button>
      </div>
    );
  };

export default FunctionalComponent;
```